



INSTITUTO SUPERIOR TÉCNICO
Universidade de Lisboa

COMPUTER CONTROL



Discrete-Time State-Variable Feedback Control

João Miranda Lemos¹ and João Pedro Gomes²

Version 2, October 2016

¹Instituto Superior Técnico, Universidade de Lisboa, Portugal, jlml@inesc-id.pt

²Instituto Superior Técnico, Universidade de Lisboa, Portugal, jpg@isr.utl.pt

Abstract

This text presents, in outline form, the main ideas behind the design of state-variable feedback controllers. It is primarily intended to support the students taking the laboratory component of the Computer Control course of the MSc in Aerospace Engineering and of the MSc in Electrical and Computer Engineering at IST, but it may be useful in other contexts as well.

Contents

Introductory note	3
1 State-space model of discrete-time systems	5
2 State available for direct measurement	5
3 LQ optimal control	6
4 Poles of the LQ controller	8
5 Predictor observer	9
6 Current observer	12
7 The Kalman filter	13
8 The LQG controller	15
9 Loop gain recovery	16
10 Symmetric root locus for the Kalman filter error poles	18
11 Tracking a non-zero reference	18
12 Transfer function of the controlled system	19
13 Final remarks	20

Introductory note

Instruments of observation have in the past led to great advances, both in Science and Technology and in Philosophy, through the way human beings — each one of us — regard their place in the World and the meaning of their lives. Two extreme examples are Galileo's astronomical telescope (a refracting telescope) and the microscope, invented in the seventeenth century by Anton van Leeuwenhoek (Holland, 1632–1723). With the astronomical telescope, Galileo observed the surface of the Moon in enough detail to show that, contrary to what the official current of thought then maintained, celestial bodies had as many imperfections as terrestrial ones. He further discovered that Jupiter, with its natural satellites, formed a sort of miniature of the solar system as described by Copernicus. Galileo was persecuted by the Inquisition but, together with Copernicus, Newton and others, contributed to a new vision of the World that radically changed human life.

Spinoza (1632–1677), the son of Portuguese Jews who had settled in Amsterdam to escape the Inquisition, lived in a part of Holland which at the time enjoyed an unusual freedom of expression — which even so did not spare him conflicts with his own Jewish community, from which he was expelled, and with a Calvinism that was gaining strength. It was certainly no accident that Spinoza took up the making of lenses for microscopes and telescopes as a livelihood, although it was through his activity as a philosopher that his name endured. Indeed, for Spinoza, Science, Philosophy and technology (optics, in this case) were not separate branches but integral parts of his life — which earned him the standing of one of the most important philosophers of all time, and led him to defend ideas that are extremely topical today, and whose value is more than ever on the agenda: the right to freedom of thought and expression, or the importance of a happy life for every human being.

Although the telescope deals with objects of enormous size (planets or galaxies), and the microscope with objects of very small size (bacteria, or viruses in the case of modern electron microscopes), both magnify detail. By contrast, the **macroscope** allows us to abstract detail away, opening the eyes of our understanding so as to grasp *complex systems* as a whole. Once the essence of the problem is understood, the absence of inessential detail makes it possible to outline a strategy for the solution. In following that strategy we must then bring the details back in — for they are often what makes the difference between working and not working.

Unlike microscopes and telescopes, which we can buy in a shop, the macroscope is not for sale in any shopping centre, near you or far from you. We can, however, build one inside ourselves, starting by adopting the right attitude towards problems and trying to look at them globally. Let us explain with an example. When the first radio communications between Europe and America were made at the beginning of the twentieth century there was a puzzle: the curvature of the Earth ought to prevent the straight-line propagation of radio waves. Although it is known today that communication was possible thanks to the ionosphere — an ionized layer surrounding the Earth that repeatedly reflects the waves — at the time this was unknown, so attention turned to diffraction, a phenomenon of wave propagation that allows waves to bend around obstacles.

It was shown, however, that diffraction could not explain the propagation of the waves

between Europe and America. Whoever did so developed a mathematical proof but, before plunging into the calculations, carried out a very simple analysis that gave a first idea of the problem: he took a sphere whose radius stood to the wavelength of light as the radius of the Earth stood to the wavelength of the radio waves used in the communication. He held the sphere up against the light and saw no diffraction at all. This reasoning is not a conclusive argument (the problem may not be scalable), but it provided a reasonable intuition about what to expect.

The macroscope works in just this way, in our minds, as a machine that constructs the essential part of a problem so as to give us a first intuition about the solution, or at least to point us towards a path that leads to one. Without being the final answer to every problem (after all, there must be some little corner left for Engineers that the prodigious victories of Artificial Intelligence do not occupy, or we should all be out of work. . .), the macroscope is — alongside the machine that answers every question (*The-Know-It-All Machine*, invented in the 1950s by the improbably brilliant inventor Gyro Gearloose) and the Swiss army knife with a built-in ballpoint pen — an essential tool that should equip every new candidate Engineer.

In the form described here, the macroscope was originally presented in 1975 by Joël de Rosnay in his book *Le Macroscopie. Vers une vision globale* (Éditions du Seuil)³. As noted in that work, the term “megaloscope” was used by the mathematician and writer of fantasy Lewis Carroll, author of the famous story *Alice in Wonderland*. It is telling that the forerunner of Ecology, Howard T. Odum, used the term “macroscope” some half a century ago, in 1971.

A good macroscope, of course, is not acquired overnight. Like good Port wine, it has to be matured over years of varied experience and care. These notes aim to give a small helping hand with the design and — crucially — the tuning of the gains of a state-variable feedback controller for the flexible beam used in the Computer Control laboratory assignment. The text sets out to explain the main points and to attempt a global perspective on the problem. The details can be found in the excellent book⁴ by Franklin, Powell and Workman, *Digital Control of Dynamic Systems*, and especially in Chapter 8 (3rd edition).

João Pedro Gomes is entirely innocent of this preface!

³A Portuguese translation was first published by Arcádia in 1977, and an American edition is available on the internet.

⁴*The Judge* is a story of the popular comic book hero Lucky Luke. It tells the tale of Judge Roy Bean in the nineteenth-century American West. As the story explains, Judge Roy Bean based his rulings on a book of civil law, which he never used, and on a Winchester, which he used all the time. One day — at least in the story in which Lucky Luke intervenes — Roy Bean is driven out of the town where he lived. With nothing to do, he started reading the Code and concluded: *This book is full of interesting things. Some of them even useful to a judge!*. The same can certainly be said of the book by Franklin and his co-authors: *This book is full of interesting things. Some of them even useful to an electrical engineer!*

1 State-space model of discrete-time systems

The process to be controlled is assumed to be described by the linear, time-invariant, finite-dimensional discrete-time state model

$$x(k+1) = \Phi x(k) + \Gamma u(k), \quad (1)$$

$$y(k) = Hx(k), \quad (2)$$

where

- $u \in \mathbb{R}, y \in \mathbb{R}$ (SISO system),
- $x \in \mathbb{R}^n$,
- $k = 0, 1, 2, 3, \dots$ (discrete time).

2 State available for direct measurement

Control law (linear state-variable feedback):

$$u(k) = -K_c x(k). \quad (3)$$

Closed-loop system ((3) $\rightarrow$ (1)):

$$x(k+1) = (\Phi - \Gamma K_c) x(k). \quad (4)$$

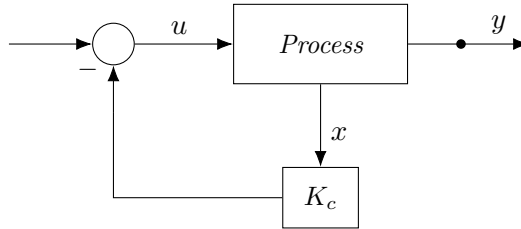


Figure 1: State feedback control.

Figure 1 shows the block diagram of this system, in which the state x is assumed to be available for direct measurement. Since this is generally not the case, other control laws in which feedback is taken from the output will be studied later.

Pole placement problem: determine the gain vector K_c such that the closed-loop dynamics matrix, $\Phi - \Gamma K_c$, has eigenvalues (that is, closed-loop poles) at specified locations $\beta_1, \beta_2, \dots, \beta_n$. In other words, letting $\alpha_c(z)$ be the desired characteristic polynomial, of the form

$$\alpha_c(z) = (z - \beta_1)(z - \beta_2) \cdots (z - \beta_n),$$

the aim is to compute the feedback gain vector K_c such that

$$\det(zI - \Phi + \Gamma K_c) = \alpha_c(z). \quad (5)$$

Equation (5) can be solved by the *method of undetermined coefficients*: equating the coefficients of the monomials of equal order on both sides of (5) yields a system of n algebraic equations in the n unknown components of K_c .

A question arises: does this system of algebraic equations always have a solution? Put differently, does equation (5) have a solution whatever the specified characteristic polynomial $\alpha_c(z)$?

The answer is given by the following theorem (which we shall not prove).

Theorem 1. *Equation (5) has a solution if and only if the matrix*

$$\mathcal{C}(\Phi, \Gamma) \triangleq \begin{bmatrix} \Gamma & \Phi\Gamma & \Phi^2\Gamma & \dots & \Phi^{n-1}\Gamma \end{bmatrix},$$

*called the **controllability matrix** associated with the state realization (Φ, Γ) , has rank n :*

$$\text{rank } \mathcal{C}(\Phi, \Gamma) = n. \quad (6)$$

□

The condition of Theorem 1 has an interpretation in terms of the structure of the system: it means that the actuator (or actuators, for multivariable systems) must be placed so as to influence every component of the state.

For *controllable systems* — that is, systems satisfying condition (6) of Theorem 1 — the gains can be computed with **Ackermann's formula**,

$$K_c = [0 \ \dots \ 0 \ 1] \mathcal{C}^{-1} \alpha_c(\Phi), \quad (7)$$

where

$$\alpha_c(\Phi) = \Phi^n + \alpha_1^c \Phi^{n-1} + \dots + \alpha_n^c I \quad (8)$$

is the desired characteristic polynomial evaluated at $z = \Phi$.

In MATLAB, Ackermann's formula is implemented by the function *acker*. This function is satisfactory for SISO systems of order up to 10 and can handle repeated roots. For higher-order systems, or for multivariable systems, the function *place* should be used instead.

3 LQ optimal control

All things are lawful for me, but not all things are profitable.

St. Paul

For controllable systems we can compute the state-variable feedback gains so as to place the closed-loop poles anywhere we wish. This, however, raises the problem of determining suitable

locations for the poles.

Clearly, the closed-loop poles must lie inside the unit circle for the closed-loop system to be asymptotically stable. Many degrees of freedom nevertheless remain.

One possibility is to design the controller so as to minimize the quadratic cost

$$J = \frac{1}{2} \sum_{k=0}^{\infty} \left[x^{\top}(k) Q x(k) + R u^2(k) \right],$$

where Q is an $n \times n$ positive semidefinite square matrix and R is a scalar, $R > 0$.

Observe that, if we choose

$$Q = H^{\top} H \tag{9}$$

then (noting that y is a scalar, so that $y^{\top} = y$)

$$x^{\top} Q x = x^{\top} H^{\top} H x = (Hx)^{\top} H x = y^{\top} y = y^2,$$

and the cost becomes

$$J = \frac{1}{2} \sum_{k=0}^{\infty} \left[y^2(k) + R u^2(k) \right].$$

This cost has two terms:

- one term (y^2) reflects the objective of keeping the output power small;
- the other term (u^2) reflects the objective of keeping the manipulated variable small.

In general these objectives conflict, and the parameter R (the “weight” on the control action) sets the relative importance of one against the other.

Thus, a small value of R means that the control term of the cost counts for less, so the controller will reduce y and let u grow. Increasing R , the power of u decreases and that of y increases (which shows, for instance, in the peak values of these variables), as illustrated in figure 2 for a typical situation.

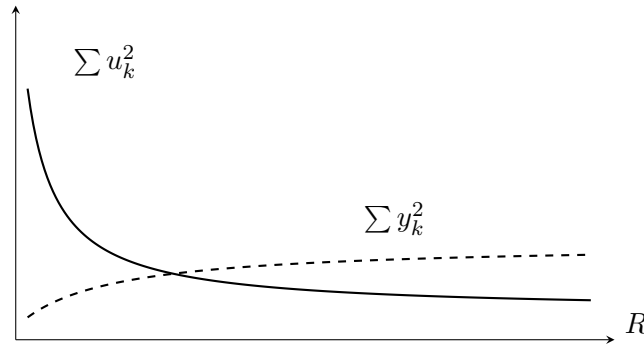


Figure 2: Typical variation of the input and output signal powers with the parameter R .

From a **frequency-domain** point of view, as R increases the closed-loop system becomes slower, that is, its bandwidth decreases. We can understand this intuitively: as R grows, the manipulated variable has less room to respond because its growth is penalized.

The parameter R is thus a very important “tuning knob” with which the designer shapes the response of the controlled system. When we reduce R , the closed-loop bandwidth increases and the response becomes faster (which is generally good), but two undesirable phenomena occur:

- the amplitude of u increases, which may drive the actuator into saturation;
- the increased bandwidth means that effects not accounted for in the model (for instance, high-frequency resonant peaks, such as those of the flexible beam used in the laboratory) get amplified (because the increased bandwidth raises the high-frequency loop gain), and these may drive the closed-loop system unstable.

The choice of R is therefore a compromise between achieving a fast response and preserving stability. Note that, in the ideal situation in which the model order used for design is correct, the LQ controller guarantees asymptotic stability of the closed-loop system. This is no longer so if the assumed model order is lower than that of the real system, in which case the high-frequency modes (very fast poles neglected in the model) may render the closed loop unstable. In the flexible beam example that motivates these notes, the real system has infinite order, corresponding to the many oscillation modes of the beam, and any finite-dimensional model neglects high-frequency modes. As the bandwidth is increased it is therefore inevitable that high-frequency modes show up.

In practice we should start with a high value of R and run experiments, or simulations, in which it is progressively lowered, in order to find the best compromise between performance and stability in the face of modelling errors (robust stability).

The control law obtained by minimizing J is called *Linear* (because the model (1), (2) is linear) and *Quadratic* (because the cost is quadratic), or **LQ control**.

The solution (which can be derived using *Dynamic Programming*) is a constant state feedback

$$u(k) = -K_{LQ} x(k), \quad (10)$$

where the optimal gain vector is

$$K_{LQ} = \frac{1}{R + \Gamma^\top S \Gamma} \Gamma^\top S \Phi, \quad (11)$$

and the matrix S satisfies the algebraic equation known as the **algebraic Riccati equation**

$$S = \Phi^\top \left[S - \frac{S \Gamma \Gamma^\top S}{R + \Gamma^\top S \Gamma} \right] \Phi + H^\top H. \quad (12)$$

In MATLAB, the LQ controller gains can be obtained with the function `dlqr`.

4 Poles of the LQ controller

Where are the closed-loop poles placed when the LQ controller is used? The answer is given by the following theorem.

Theorem 2 (Chang, Letov). *The closed-loop poles of the LQ controller (that is, the eigenvalues of the matrix $\Phi - \Gamma K_{LQ}$, with K_{LQ} computed from (11)) are the n stable roots (inside the unit circle) of the algebraic equation*

$$1 + \frac{1}{R} G(z)G(z^{-1}) = 0,$$

where

$$G(z) = H(zI - \Phi)^{-1}\Gamma$$

is the transfer function of the system to be controlled, corresponding to model (1), (2), or, equivalently, of

$$R a(z)a(z^{-1}) + b(z)b(z^{-1}) = 0, \quad (13)$$

where

$$\frac{b(z)}{a(z)} = G(z).$$

□

Note that, for very small R , (13) reduces to

$$b(z)b(z^{-1}) = 0,$$

so that, for this choice of R , the closed-loop poles cancel the zeros if these lie inside the unit circle, or are their “mirror images” (that is, a zero at z_i gives rise to a pole at $1/z_i$) if the zeros lie outside the unit circle.

Likewise, if R is very large, (13) becomes

$$a(z)a(z^{-1}) = 0.$$

In this case the closed-loop poles coincide with the open-loop poles if these are stable, or with their mirror images if the open-loop poles are unstable.

In either case, the closed-loop system is always asymptotically stable when the LQ controller gains are used.

For intermediate values it is necessary to solve equation (13) and take the n stable poles among its $2n$ roots. In this way we learn the location of the closed-loop poles corresponding to the gain K_{LQ} . This technique is known as the **symmetric root locus**.

5 Predictor observer

In most situations we cannot measure the state x directly, but only the output y , related to it through (2), which represents the sensor model. One strategy is then to estimate the state from the available signals (u and y) and the system model, and to feed this estimate back using the LQ controller gains. This is shown in figure 3, where the state estimate is denoted by $\hat{x}$. This approach raises two questions:

1. How should the state be estimated?
2. Can the state estimator and the controller (with the LQ gain or any other state feedback) be designed separately, and will the system resulting from coupling the two (estimator and controller) still have its poles at the specified locations?

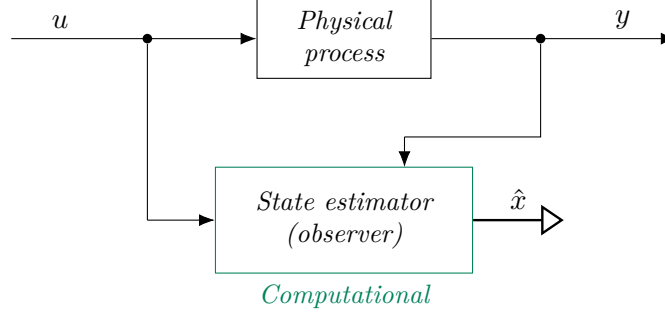


Figure 3: State estimator (observer).

How can we define a state estimation algorithm (also called an **observer**) that lets us build, as shown in figure 4, an estimate $\hat{x}$ (a function of time k) of the state x from the observations of successive samples of u and y and from knowledge of the state model (1), (2)?

Figure 4 illustrates what we are after: initially the state estimate $\hat{x}(0)$ may be very different from the state $x(0)$. As discrete time advances we want the estimate $\hat{x}(k)$ to approach the true value of the state $x(k)$.

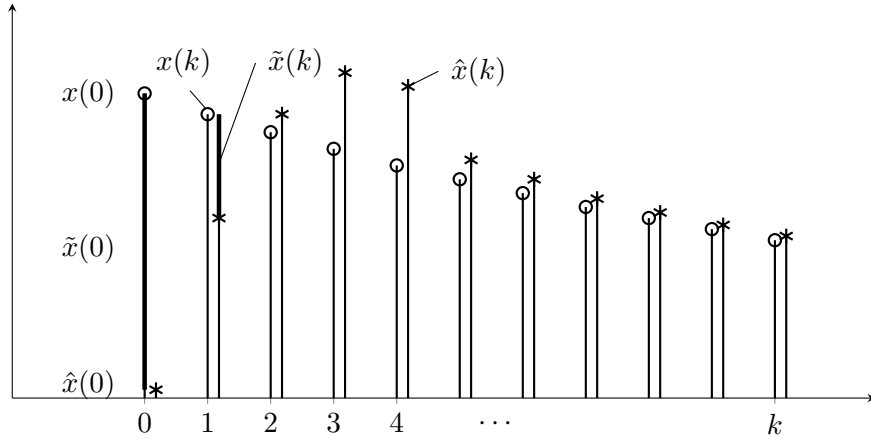


Figure 4: The state x , its estimate $\hat{x}$ and the estimation error $\tilde{x} = x - \hat{x}$.

A first possibility would be, as shown in figure 5, to “build” (integrate the equations of) a replica of the system and drive it with the same input that is driving the system.

The equation defining the observer would be a replica of the state equation modelling the system, driven by the same input but with a different state (the estimate $\hat{x}$), giving (compare with equation (1))

$$\hat{x}(k+1) = \Phi\hat{x}(k) + \Gamma u(k). \quad (14)$$

The state estimation error, $\tilde{x} = x - \hat{x}$, satisfies the equation obtained by subtracting (14)

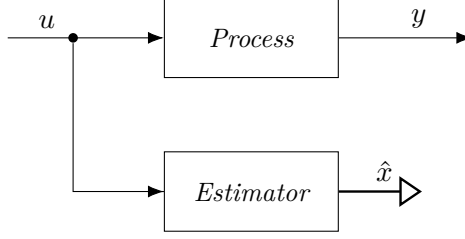


Figure 5: Open-loop observer.

from (1), namely the difference equation

$$\tilde{x}(k+1) = \Phi \tilde{x}(k), \quad (15)$$

with initial condition

$$\tilde{x}(0) = x(0) - \hat{x}(0). \quad (16)$$

The initial value problem (15), (16) is readily solved, giving the following expression for the error as a function of time k :

$$\tilde{x}(k) = \Phi^k \tilde{x}(0). \quad (17)$$

It follows at once that, for the error to tend to zero, all eigenvalues of Φ must lie inside the unit circle, that is, the open-loop process must be asymptotically stable.

With the state estimator of equation (14) it is therefore impossible to estimate the state of an unstable process, since the error would diverge. Even for asymptotically stable processes, the rate at which the error converges to zero equals the rate of the transients of the very process we wish to control (because in both cases, process and observer, the dynamics is defined by the same matrix Φ), which prevents the state from being known accurately during the transient. The estimator (14) is thus useless — but it can be modified.

Another possibility is therefore to add, to the equation defining the estimator, a term depending on the measured process output:

$$\hat{x}(k+1) = \Phi \hat{x}(k) + \Gamma u(k) + L_p (y(k) - H \hat{x}(k)), \quad (18)$$

where $L_p \in \mathbb{R}^n$ is a vector to be designed. The additional term is proportional to $y(k) - H \hat{x}(k)$, that is, to the difference between what we observe at the output at time k , $y(k)$, and what we expect to observe given the state estimate and our knowledge of the observation model defined by (2).

The error equation is obtained by subtracting (18) from (1), and is now

$$\tilde{x}(k+1) = \Phi \tilde{x}(k) - L_p (y(k) - H \hat{x}(k)).$$

Taking (2) into account, the error dynamics equation is

$$\tilde{x}(k+1) = (\Phi - L_p H) \tilde{x}(k). \quad (19)$$

The observer gain L_p is designed so as to impose the eigenvalues of $\Phi - L_p H$. Letting $\alpha_o(z)$ be the characteristic polynomial specified for the observer, L_p satisfies

$$\det(zI - \Phi + L_p H) = \alpha_o(z). \quad (20)$$

In analogy with the state-feedback pole placement problem, we have the following theorem.

Theorem 3. *Equation (20) has a solution if and only if the matrix*

$$\mathcal{O}(H, \Phi) \triangleq \begin{bmatrix} H \\ H\Phi \\ H\Phi^2 \\ \vdots \\ H\Phi^{n-1} \end{bmatrix},$$

*called the **observability matrix**, has rank n :*

$$\text{rank } \mathcal{O}(H, \Phi) = n. \quad (21)$$

□

As in the controller case, equation (20) can be solved for small problems by the method of undetermined coefficients. It can also be solved by Ackermann's formula (7) (using the functions *acker* or *place*) with Φ replaced by $\Phi^\top$ and Γ replaced by $H^\top$.

6 Current observer

Observer (18) is such that $\hat{x}(k)$ is computed from the output observation $y(k-1)$ made at the previous instant. For this reason it is called a **predictor observer**. It has the drawback of not using the most recent information.

The alternative is to use the so-called **current observer**, defined by the recursion

$$\hat{x}(k) = \Phi \hat{x}(k-1) + \Gamma u(k-1) + L_c [y(k) - H(\Phi \hat{x}(k-1) + \Gamma u(k-1))]. \quad (22)$$

An alternative way of writing the observer equations is

$$\hat{x}(k) = \bar{x}(k) + L_c (y(k) - H\bar{x}(k)), \quad (23)$$

$$\bar{x}(k+1) = \Phi \hat{x}(k) + \Gamma u(k), \quad (24)$$

where $\bar{x}$ is the predicted estimate and $\hat{x}$ is the current estimate.

Equations (23) and (24) may be interpreted as follows:

1. We are at time k and have available a predicted estimate $\bar{x}(k)$, made at the previous instant $k-1$.

2. We take an observation $y(k)$ and correct the estimate $\bar{x}$ on the basis of this observation to obtain the current estimate at time k , $\hat{x}(k)$, using (23).
3. From the current estimate at time k , $\hat{x}(k)$, and using (24), we compute a predicted estimate for the next instant, $\bar{x}(k+1)$, and iterate the procedure by returning to step 1 with k replaced by $k+1$.

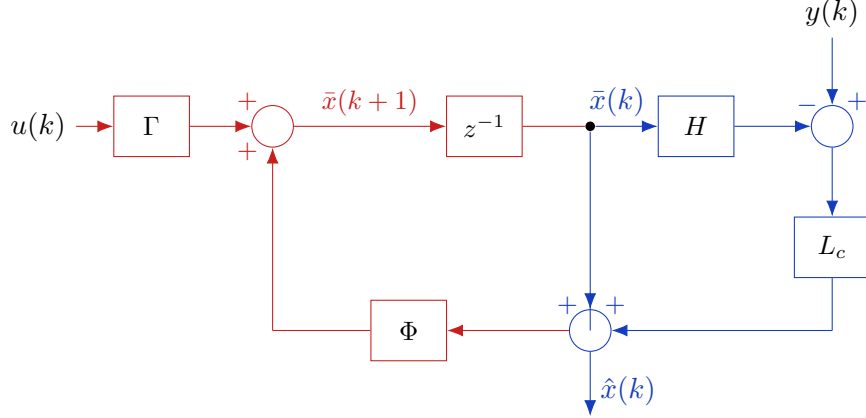


Figure 6: Block diagram of the current observer. In red, equation (24); in blue, equation (23).

Equations (22) on the one hand, and (23) and (24) on the other, are perfectly equivalent. This is readily checked by replacing k with $k-1$ in (24) and substituting the resulting expression for $\bar{x}(k)$ into (23), which yields (22).

The advantage of using the pair (23), (24) is that it corresponds to a block diagram involving no explicit delay in $u(k)$, unlike equation (22). Moreover, (23) and (24) can be represented by the block diagram of figure 6. In that figure the red part corresponds to equation (24) and the blue part to equation (23). This block diagram has $y(k)$ and $u(k)$ as inputs and the current state estimate at time k , $\hat{x}(k)$, as output, although the predicted estimate is available as well.

The current observer gain L_c is related to the predictor observer gain L_p by

$$L_c = \Phi^{-1} L_p.$$

We may therefore compute L_p using Ackermann's formula and then obtain L_c from the expression above. We shall, however, see next another, optimized way of designing L_c .

7 The Kalman filter

The Kalman filter⁵ is a current observer in which the state estimate is therefore propagated in time according to equation (22), but in which the observer gain L_c is computed so as to optimize the signal-to-noise ratio.

⁵The Kalman filter is one of the most important discoveries of Electrical and Computer Engineering. It was conceived in the early 1960s by Rudolf Kalman in the discrete-time version, and by R. Kalman and Richard S. Bucy in the continuous-time version.

In the frequency domain, the choice of the desired eigenvalues for the error equation can be seen as follows: choosing the eigenvalues implies a value for the observer bandwidth. The faster the specified eigenvalues (that is, the closer they are to the origin), the larger the observer bandwidth.

One might think that increasing the observer bandwidth is desirable, since it leads to a better reproduction of the time evolution of the state, but in fact this is not so, for two reasons:

1. There may be modelling errors at high frequency. Increasing the observer bandwidth amplifies these errors and may cause the observer to diverge (that is, the error grows instead of decreasing).
2. There are random disturbances (“system noise”) and sensor noise.

Let us consider point 2) in more detail. In the presence of noise the state model becomes

$$x(k+1) = \Phi x(k) + \Gamma u(k) + v_1(k), \quad (25)$$

$$y(k) = Hx(k) + v_2(k). \quad (26)$$

The signals v_1 (a vector) and v_2 (a scalar) are uncorrelated Gaussian sequences with zero mean and variance

$$E[v_1(k)v_1^T(k)] = Q_E, \quad (27)$$

$$E[v_2^2(k)] = R_E. \quad (28)$$

Assuming this model to be valid, consider a situation corresponding to a given value of the observer gain L_c and, correspondingly, to a given observer bandwidth. In this situation the observer “lets through” to the estimate $\hat{x}$ those components of the state x that lie within its band. And of course it lets noise through as well.

Redesigning L_c so as to increase the observer bandwidth lets more components of the state x through (which is good), but also lets more noise through, in proportion to the increase in bandwidth (which is bad).

Increasing the observer bandwidth progressively (by manipulating the gain L_c), there comes a point beyond which no further frequencies of the signal of interest x are let through (assuming this signal is low-pass), but more and more noise frequencies still are (since, the noise being white, its power spectral density is constant in frequency). We may then ask: what observer bandwidth (and corresponding distribution of the observer poles) optimizes the signal-to-noise ratio?

The **Kalman filter** answers a slightly different, but related, question. Consider for that purpose the cost

$$J_o = E \left[\sum_{k=0}^{\infty} \|x(k) - \hat{x}(k)\|^2 \right].$$

Which unbiased estimator (that is, one for which $E[x(k) - \hat{x}(k)] = 0, \forall k$) minimizes J_o ?

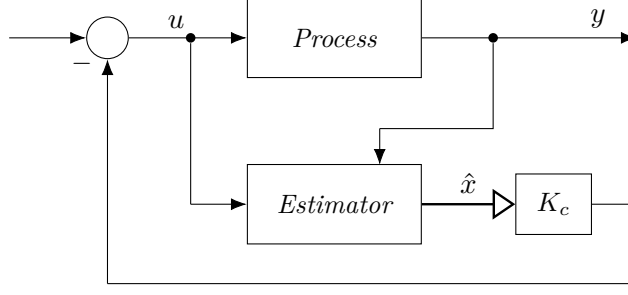


Figure 7: Structure of the LQG controller.

When the state and the observations are generated by model (25), (26), the requirement that the estimator be unbiased leads to the observer structure (equation (22)).

Minimizing J_o leads to a gain given by

$$L_c = \Phi P H^\top \frac{1}{R_E + H P H^\top}, \quad (29)$$

where the state estimation error covariance matrix P satisfies the algebraic Riccati equation

$$P = \Phi P \Phi^\top + Q_E - \frac{\Phi P H^\top H P \Phi^\top}{R_E + H P H^\top}. \quad (30)$$

The optimal gain vector corresponding to the Kalman filter (that is, to the optimized current observer) can be computed in MATLAB with the function *dlqe*.

8 The LQG controller

We can couple an LQ controller to an observer, feeding back the state estimate, as shown in figure 7, to obtain the so-called LQG (Linear Quadratic Gaussian) controller. Here we are dealing with the so-called **regulation problem**, in which the reference to be tracked is zero. Non-zero references (the servomechanism problem) will be considered later.

The equations defining the controller correspond to the block diagram of figure 7 and are therefore

$$\hat{x}(k) = \Phi \hat{x}(k-1) + \Gamma u(k-1) + L_c [y(k) - H (\Phi \hat{x}(k-1) + \Gamma u(k-1))], \quad (31)$$

$$u(k) = -K_c \hat{x}(k). \quad (32)$$

The current observer gain L_c and the controller gain K_c are computed as explained in the previous sections. In particular, L_c can be obtained from the Kalman filter expressions or with the MATLAB function *dlqe*, and K_c can be obtained from the LQ controller expressions or with the MATLAB function *dlqr*.

Note that we can compute the transfer function of the LQG controller, relating the controller input y (equal to the process output) to the negative of the controller output, $-u$ (the input of the process to be controlled), yielding the block diagram of figure 8, which represents the interconnection of process and controller.

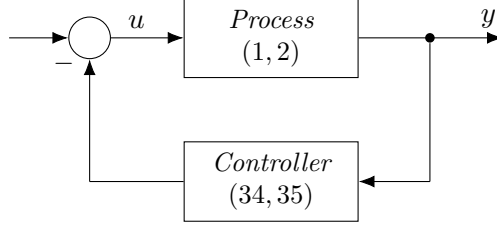


Figure 8: The LQG controller represented by its transfer function.

Indeed, eliminating the variable u in equation (31) that defines the observer, using the control law (32) and regrouping terms, we get

$$\hat{x}(k) = (\Phi - \Gamma K_c - L_c H \Phi + L_c H \Gamma K_c) \hat{x}(k-1) + L_c y(k). \quad (33)$$

Note that (33) relates $\hat{x}(k)$ to $\hat{x}(k-1)$ and to $y(k)$ — not to $y(k-1)$ — precisely because the current observer uses the most recent observation. To obtain a controller state model in the standard form $x(k+1) = Ax(k) + Bu(k)$, $y(k) = Cx(k) + Du(k)$, which is the form required by the MATLAB functions used in section 9, it is convenient to take the predicted estimate $\bar{x}$ as the controller state. From (23), (24) and (32) we then obtain

$$\bar{x}(k+1) = (\Phi - \Gamma K_c)(I - L_c H) \bar{x}(k) + (\Phi - \Gamma K_c) L_c y(k), \quad (34)$$

$$-u(k) = K_c(I - L_c H) \bar{x}(k) + K_c L_c y(k). \quad (35)$$

Equations (34) and (35) define a state model for the controller in which the dynamics matrix is $(\Phi - \Gamma K_c)(I - L_c H)$, the input vector is $(\Phi - \Gamma K_c)L_c$, the input is y (the process output), the output vector is $K_c(I - L_c H)$, the direct feedthrough is $K_c L_c$, and the output is $-u(k)$ (the negative of the process input).

9 Loop gain recovery

Recall that the loop gain is the product of all transfer functions around the feedback loop. As can be seen from figure 7, the loop gain is easily computed in MATLAB as the product of the transfer functions of the process (defined by the state equations (1) and (2)) and of the controller (defined by the state equations (34) and (35)). The procedure for computing the loop gain in MATLAB is as follows:

1. Convert the system state model (defined by the state equations (1) and (2)) to an equivalent transfer function, using the MATLAB function *ss2tf*.
2. Convert the controller state model (defined by the state equations (34) and (35)) to an equivalent transfer function, using the MATLAB function *ss2tf*.
3. Use the MATLAB function *series* to obtain the transfer function of the product of the two transfer functions above, which gives the loop gain.

4. Use the function *bode* to plot the frequency response of the loop gain. Note: make sure to plot over a suitable frequency range.

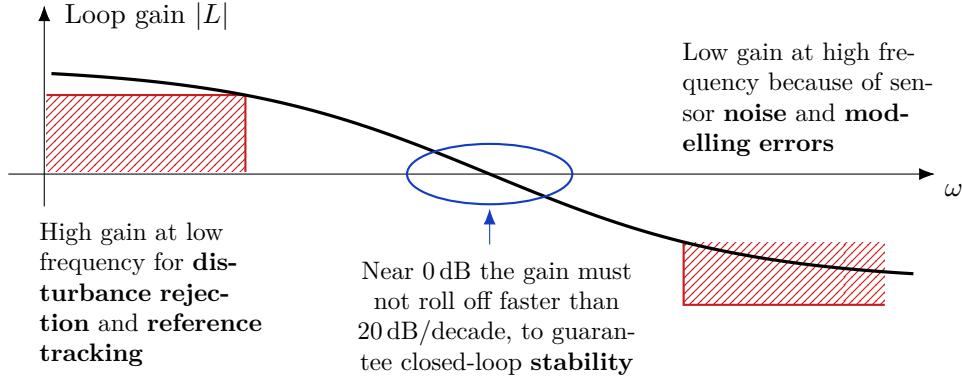


Figure 9: Constraints on the shaping of the loop gain in a feedback control loop.

As studied in the basic control course, the loop gain plays a role of great importance in the design of feedback controllers.

As shown in figure 9, the controller design must “shape” the loop gain so as to meet the following constraints:

1. High loop gain at low frequencies, to guarantee attenuation of disturbances and tracking of references.
2. Low loop gain at high frequencies, to guarantee insensitivity to sensor noise (which appears at high frequency) and the robust stability condition (that is, the unmodelled high-frequency dynamics must be multiplied by a gain small enough to prevent it from destabilizing the closed-loop system).
3. In the vicinity of 0 dB the rate of decay of the loop gain with frequency must not be too large (in magnitude), so as not to cause instability, ensuring that the system has adequate relative stability margins (phase margin and gain margin)⁶.

When direct measurement of the state is available, the LQ controller provides good relative stability margins (infinite gain margin in many cases, and 60° phase margin).

However, when state feedback is replaced by the estimate $\hat{x}$ provided by a Kalman filter, these relative stability guarantees vanish completely. Depending on the noise covariance matrices Q_E and R_E in model (25), (26), the relative stability margins may be nearly zero.

The idea then arose⁷ of choosing Q_E and R_E not on the basis of the noise covariances (which are very difficult to measure), but by regarding these matrices as parameters that the designer can tune. In particular, it is possible to choose Q_E and R_E in such a way that the loop gain of the LQG controller approaches that of the LQ controller (with its good relative stability properties). This technique is called loop gain recovery, or LTR (*Loop Transfer Recovery*).

⁶This is a consequence of Bode’s integral theorem, which relates phase to the slope of the magnitude curve for a minimum-phase system. According to this theorem, the steeper the slope of the magnitude frequency response, the smaller the phase — which implies a reduction of the phase margin.

⁷This idea appeared around 1979.

In the Computer Control laboratory assignment, a choice of Q_E and R_E is suggested that amounts to setting $R_E = 1$ and $Q_E = qI$ with q a large number. This is only an initial suggestion, and can be optimized.

10 Symmetric root locus for the Kalman filter error poles

The influence of the weight on the Kalman filter error dynamics can be computed using a technique similar to the one used for the control weight, as studied in section 4. In fact, it can be shown (Franklin *et al.*, 3rd ed., section 9.4.3, p. 396) that, for a SISO system, the poles of the estimation error dynamics with the Kalman filter follow the same pattern as the optimal poles of the LQ controller. Specifically, the optimal poles satisfy

$$1 + \gamma G_e(z^{-1})G_e(z) = 0, \quad (36)$$

where

$$\gamma = \frac{q_E}{R_E} \quad (37)$$

and

$$G_e(z) = H(zI - \Phi)^{-1}\Gamma_e. \quad (38)$$

In (38), Γ_e is the vector through which the process noise acts on the state, that is, $v_1(k) = \Gamma_e w(k)$, where w is a scalar of variance $E[w^2(k)] = q_E$, so that $Q_E = q_E \Gamma_e \Gamma_e^T$. If the process noise enters through the input channel, then $\Gamma_e = \Gamma$.

11 Tracking a non-zero reference

So far we have been concerned with the so-called regulation problem, which consists in driving the output to zero in the face of non-zero initial conditions or disturbances. We now consider the problem of tracking a non-zero reference (the so-called **servomechanism** problem).

There are several ways of solving the servomechanism problem, corresponding to closed-loop systems with different zeros (and hence different dynamic responses). In the solution illustrated in figure 10, the reference signal r can be introduced through the control law

$$u(k) = \bar{N}r - K_c \hat{x}(k), \quad (39)$$

where $\bar{N}$ is a scalar such that the static gain of the transfer function equals 1, which — together with asymptotic stability of the closed loop — guarantees that, in steady state, $y \approx r$.

$\bar{N}$ can be computed from the formulas

$$\begin{bmatrix} N_x \\ N_u \end{bmatrix} = \begin{bmatrix} \Phi - I & \Gamma \\ H & 0 \end{bmatrix}^{-1} \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad (40)$$

$$\bar{N} = N_u + K_c N_x. \quad (41)$$

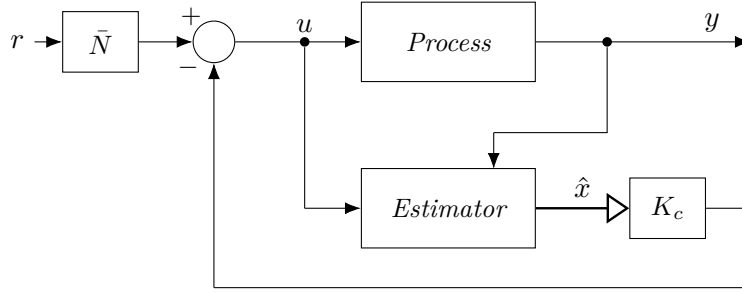


Figure 10: Introducing the reference.

The derivation can be found in

Franklin, Powell, Workman. *Digital Control of Dynamic Systems*, 3rd ed., Addison Wesley, pp. 310–313.

Another possibility is to introduce the reference in such a way that the tracking error is amplified. The details of this approach are omitted here and can be found in the book cited above.

12 Transfer function of the controlled system

The transfer function of the controlled system expresses the relation between the reference and the output (the position of the beam tip). It is very important to look at the frequency response corresponding to this transfer function, since it gives precise information about whether the specifications are met, for instance:

- Is the static gain equal to 1 (that is, does the output track a constant reference in steady state)? The low-frequency gain of the transfer function of the controlled system must be 0 dB (1 in linear units).
- Is the bandwidth adequate?
- Is there peaking in frequency?

The transfer function of the controlled system is easily obtained in MATLAB with the following procedure:

1. Convert the system state model (defined by the state equations (1) and (2)) to an equivalent transfer function, using the MATLAB function *ss2tf*.
2. Convert the controller state model (defined by the state equations (34) and (35)) to an equivalent transfer function, using the MATLAB function *ss2tf*.
3. Use the MATLAB function *feedback* to obtain the closed-loop transfer function from the two transfer functions above.

4. Use the function *series* again to obtain the transfer function of the controlled system as the series connection of the transfer function obtained in the previous step with the gain $\bar{N}$.
5. Use the function *bode* to plot the frequency response of the transfer function of the controlled system. Note: make sure to plot over a suitable frequency range.

13 Final remarks

Figure 11 shows the SIMULINK block diagram for simulating the controlled system. In this SIMULINK diagram:

- The yellow block is the process model used to simulate the controller. In the laboratory this block is replaced by two data input and output blocks that interface with the real system. The matrices A , B and C define the state model of the linearized system. In this text they correspond, respectively, to the matrices Φ , Γ and H of the model described by equations (1) and (2). The matrix D has all entries equal to zero.
- The green blocks correspond to the current observer, implemented with the structure of figure 6.
- The red block is the state feedback gain vector.
- The blue block corresponds to the gain $\bar{N}$ computed from (41), denoted **Nbar** in this SIMULINK diagram.

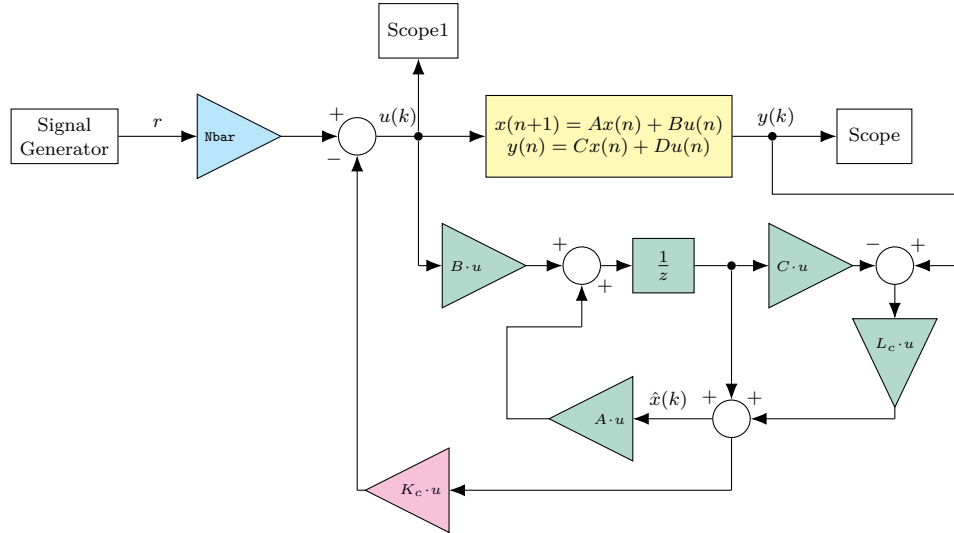


Figure 11: SIMULINK simulation of the state-feedback controller with a current observer.

The MATLAB code for designing the controller together with the current observer is shown below.

```
% Computer Control
% Design of the current observer and state feedback gains
```

```

% A, B, C, D denote the state model of the process to control
% A, B, C are obtained from model identification; D = 0

% regulator gain computation using LQ expressions
Q = C'*C;
R = 500;          % 100
[K,S,E] = dlqr(A,B,Q,R);
% dstep(A-B*K,B,C,D);

% estimator gain computation using Kalman filter expressions
w2 = 100;
QE = eye(size(A))*w2;
RE = 1;
G = eye(size(A));
[M,P,Z,EE] = dlqe(A,G,C,QE,RE);

% Computing reference feedforward gain according to
% (8.73) Franklin DCDS 3rd Ed

N = inv([A-eye(size(A)), B; C, 0])*[zeros(size(A,1),1);1];
Nx = N(1:end-1,:);
Nu = N(end,:);
Nbar = Nu + K*Nx;

```